

课程笔记

LightRAG 进阶：KG-based RAG 与知识图谱可视化

五道口纳什 & Codex

2026 年 3 月 28 日

LightRAG 进阶
KG-based RAG
KG合并/可视化
Hybrid/Mix mode

视频作者/频道：五道口纳什

发布日期：2025

视频时长：约 12 分钟

视频链接：未填写

目录

1 引言	2
2 为什么需要 KG-based RAG	2
2.1 Naive RAG 的局限	2
2.2 KG-based RAG 的优势	2
2.3 本章小结	3
3 增量索引 (Mode) 过程	3
3.1 新文档的处理流程	3
3.2 实体对齐与命名一致性	3
3.3 描述合并策略: MapReduce	3
3.4 合并示例	4
3.5 本章小结	4
4 知识图谱可视化	4
5 检索模式详解: Naive、Hybrid、Mixed	4
5.1 三种检索模式	4
5.2 关键词提取: Low-Level 与 High-Level	5
5.3 Local Search (局部搜索)	5
5.4 Global Search (全局搜索)	5
5.5 本章小结	5
6 总结与延伸	5
6.1 拓展阅读	6

1 引言

本期是 LightRAG 的进阶篇，紧接上期内容，深入介绍 LightRAG 的源码细节和工作机制。主要覆盖四个方面：

1. 为什么要引入 Knowledge Graph Base 的 RAG
2. LightRAG 的增量索引 (Mode) 过程
3. Knowledge Graph 的可视化
4. Hybrid/Mix 检索模式详解

2 为什么需要 KG-based RAG

2.1 Naive RAG 的局限

传统的 Chunk-based (Naive) RAG 存在两个核心局限：

Naive RAG 的两大痛点

- **多跳推理困难**：当实体关系跨越多个 Chunk ($A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$)，Naive RAG 只能检索到局部片段，无法串联完整的推理链
- **全局性查询缺失**：受限于 TopK 检索，如果关键 Chunk 不在 TopK 中，推理就会失败

2.2 KG-based RAG 的优势

Knowledge Graph Base 的 RAG 从数据形态到推理方式都与 Naive RAG 有本质区别：

维度	Naive RAG (Chunk-based)	KG-based RAG
数据形态	点 (每个 Chunk 一个 Embedding)	网 (实体 + 关系组成 Graph)
语义丰富度	原始文本	实体/关系有详细描述和 Embedding
推理方式	检索 Chunk $\rightarrow$ 放入 Context $\rightarrow$ 碎片化推理	图结构游走 + 关联发现 $\rightarrow$ 结构化聚合
全局性	受限于 TopK	通过图游走实现全局搜索
可解释性	较低	高 (路径可追溯)

表 1: Naive RAG 与 KG-based RAG 的核心对比

KG 的构建完全依赖语言模型

LightRAG 中的实体/关系提取和描述生成 (Profiling) 全部基于语言模型的 Prompt 完成。这意味着 KG 的质量直接取决于 LLM 的能力。

2.3 本章小结

KG-based RAG 通过将文档转化为结构化的知识图谱，实现了多跳推理和全局性查询，弥补了传统 Chunk-based RAG 在复杂推理场景下的不足。

3 增量索引 (Mode) 过程

3.1 新文档的处理流程

LightRAG 支持动态新增文档，其 Mode (Upsert = Update + Insert) 过程如下：

1. **实体关系提取**：基于语言模型从新文档中提取实体 (Entity) 和关系 (Edge)
2. **Profiling**：为实体和关系生成详细的文本描述
3. **Embedding**：为实体描述、关系描述和 Chunk 文本建立向量索引
4. **子图合并**：将新文档的子图合并到全局知识图谱中

3.2 实体对齐与命名一致性

实体命名一致性是关键

不同 Chunk 中可能用不同方式提到同一个实体。LightRAG 在 Entity Extraction System Prompt 中明确规定：

- 实体名称大小写不敏感
- 每个有意义的单词要大写（如“Knowledge Graph”而非“knowledge graph”）
- 确保一致性命名（相同实体使用相同名称）

实体对齐主要依赖语言模型生成时的一致性，而非后处理匹配。

3.3 描述合并策略：MapReduce

当同一实体或关系出现在多个 Chunk 中时，每个 Chunk 贡献了该实体/关系的一个侧面描述。合并策略采用 MapReduce 模式：

1. **收集**：将所有 Chunk 中提取到的描述汇总
2. **判断长度**：
 - 描述较短 → 用分隔符直接拼接
 - 描述较长 → 基于语言模型做语义压缩 (Summarization)
3. **同步更新**：src_id 也做 concatenation，记录描述来源

语义压缩 Prompt

LightRAG 的 prompts.py 中定义了 summarize_entity_descriptions 这一 Prompt，专门用于将过长的实体/关系描述压缩为更精练的摘要，同时保留核心信息。

3.4 合并示例

以两个 Chunk 为例：

- **Chunk 1**：提取出实体 Alex（描述 A）、Bob（描述 A）以及关系 Alex→Bob = ”同事”
- **Chunk 2**：提取出实体 Alex（描述 B）以及关系 Alex→Bob = ”朋友”

合并后：

- Alex 的描述 = 描述 A <sep> 描述 B（侧面叠加）
- Alex→Bob 的关系描述 = ”同事” <sep> ”朋友”
- 如果描述总长度超过阈值，触发语义压缩

3.5 本章小结

LightRAG 的增量索引通过实体对齐、描述合并（MapReduce）和语义压缩三步，实现了新文档的无缝融入。整个过程高度依赖语言模型的能力。

4 知识图谱可视化

LightRAG 在索引过程中会生成一个 GraphML 格式的图谱文件。项目提供了基于 pyvis 库的可视化代码：

- 点击节点可查看实体的详细描述
- 边上展示关系的描述信息
- 适合用于调试和理解 KG 的结构

GraphML 格式

GraphML 是一种基于 XML 的图结构描述格式，被广泛用于知识图谱的存储和交换。LightRAG 在 Index 阶段自动生成此文件，无需额外配置。

5 检索模式详解：Naive、Hybrid、Mixed

5.1 三种检索模式

LightRAG 支持三种检索模式（Query Mode），各有适用场景：

三种检索模式对比

- **Naive**：仅基于 Vector 检索——对 Query 做 Embedding，与 Chunk Embedding 计算 Cosine Similarity，返回 TopK
- **Hybrid**：纯粹基于知识图谱——结合 Local Search（实体邻域）和 Global Search（关系匹配），不直接查向量库中的原始文本块
- **Mixed**：Hybrid + Naive——既有 KG 的全局推理，又有传统向量检索获取原始文本块

5.2 关键词提取：Low-Level 与 High-Level

LightRAG 的检索首先从用户 Query 中提取两类关键词：

- **Low-Level 关键词**：具体的实体名称（如人名、产品名），用于 Local Search
- **High-Level 关键词**：宏观主题、抽象概念，用于 Global Search

这一步同样由语言模型完成，可通过 LangFuse 的 Trace 监控其输入输出。

5.3 Local Search（局部搜索）

1. 使用 Low-Level 关键词在实体向量库中做 Vector Search
2. 检索到对应的实体节点
3. 在 Graph 中做一跳遍历（Traverse），获取直接关联的邻居实体
4. 返回一跳范围内的实体和关系信息作为 Context

Local Search 以实体为中心，适合回答关于特定实体的详细问题。

5.4 Global Search（全局搜索）

1. 使用 High-Level 关键词在关系向量库中做 Vector Search
2. 检索到对应的关系（边）
3. 返回这些关系及其关联的实体作为 Context

Global Search 以关系为中心，关注宏观主题，适合回答跨领域、跨文档的综合性问题。

5.5 本章小结

LightRAG 通过 Low-Level/High-Level 关键词分离和 Local/Global 双重搜索，实现了从具体实体到宏观主题的多层次检索。Mixed 模式则在此基础上补充原始文本块，提供最全面的信息覆盖。

6 总结与延伸

本期深入剖析了 LightRAG 的四个核心技术细节：

1. **KG-based RAG 的必要性**：解决了 Naive RAG 在多跳推理和全局查询上的根本局限
2. **增量索引机制**：通过 MapReduce 式的描述合并和语义压缩，支持动态新增文档
3. **KG 可视化**：基于 pyvis 提供直观的图谱浏览工具
4. **多模式检索**：Naive/Hybrid/Mixed 三种模式覆盖不同的查询需求

6.1 拓展阅读

- LightRAG 源码中的 `prompts.py`: 包含所有关键 Prompt (Entity Extraction、Summarization 等)
- LangFuse 本地部署: 用于监控 LightRAG 所有 API 调用的输入输出
- Microsoft GraphRAG: 与 LightRAG 类似的 KG-based RAG 方案, 可做对比研究